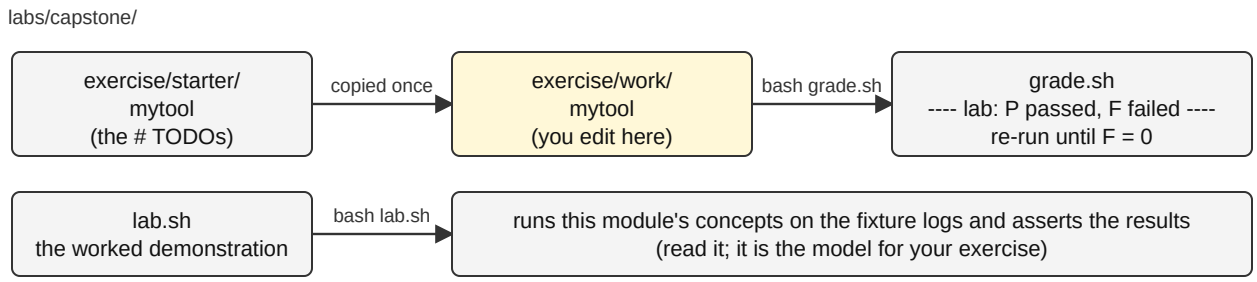
Integrative Practical A-10 — Assemble Your Own Tool: mytool (Instructor Version)

Instructor Note: Gray boxes and answers marked in red appear in the instructor copy only.

Topology



Environment Table

All paths are relative to `labs/capstone/exercise/`.

File	What it is	You edit it?
`TASK.md`	The short statement of the practical	No
`starter/mytool`	The skeleton of the finished tool with `# TODO` markers in the order to fill them	No — the grader copies it
`work/mytool`	**Your** copy (created by the first `grade.sh` run)	**Yes**
`grade.sh`	The rubric grader: 12 checks	No
`work/DECISIONS.md`	Your decisions note (≈150 words, three answers)	**Yes** (you create it)
`work/GRADER.txt`	Your final grader output, saved with `bash grade.sh > work/GRADER.txt 2>&1`	**Yes** (you create it)

Objectives

- **Part 1:** Prepare the exercise folder and watch the starter leak a temp file per run.
- **Part 2:** Assemble `mytool`: trap, usage, `getopts`, subcommand check, file-or-stdin, `case` dispatch.
- **Part 3:** Verify with the grader and with `shellcheck`.
- **Part 4:** Write the decisions note.

Background / Scenario

No new concepts. This practical asks you to assemble everything modules M00–M08 taught into one small, real command-line tool, built the way every tool in this course — and every `ci/*.sh` script in the SecMLOps program — is built. `mytool` is a tiny line-analyser:

- `mytool count [FILE]` — the number of lines, from FILE or from stdin when FILE is omitted

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

- `mytool [-n N] top [FILE]` — the most frequent line(s); `-n N` shows the top N (default 1)
- `mytool -h` — usage on **stdout**, exit 0

The standard it must meet: strict mode and a **temp-file cleanup trap** that preserves the exit code (M07); `getopts` for `-n`/`-h` with your own error handling and a `case` dispatcher for the subcommand (M06, M02); input from a **FILE or stdin** (M06); **distinct exit codes** — `0` ok, `1` unreadable file, `2` usage error or unknown command (M06); **shellcheck-clean** (M00, M08).

Time. 60 minutes, self-timed: about 45 to build, 15 for the note. You may use the module pages, any documentation, and AI assistance **for documentation lookup only** — say so in the note. Not allowed: `solution/`, or another learner's `mytool`.

Required Resources

- A lab session or a shell where `bash tools-check.sh` reports `ready` (with `shellcheck`).
- Modules M02, M05, M06, M07 (S07.3–S07.4, S06.2, S06.4–S06.5, S02.3, S05.3 are the steps the TODO markers point at).

> **Instructor Note:** **Running the assessment.** Collect three files: `work/mytool`, `work/DECISIONS.md`, `work/GRADER.txt`. Re-run the grader yourself on their copy — never trust the pasted output alone: `GRADE_TARGET=/path/to/their/work bash labs/capstone/exercise/grade.sh`. If shellcheck is missing on *your* machine the last check reports a skipped PASS; certify R-10.6 only where shellcheck runs. If the learner ran out of time: score what exists at 60 minutes for the check-backed criteria and let them finish the note afterwards (the note is reflective, not timed); if the result is Not yet, remediate and issue **A-10R**, never a second attempt at the same `mytool` with the solution in hand. Observed: the starter scores `---- capstone: 2 passed, 10 failed ----`; the reference 12/12.
>
> Watch for two things during the build: learners who edit `grade.sh` (never acceptable — compare with the course copy), and learners who paste a whole solution from an AI tool (allowed only for documentation lookup; the note is where you find out). The observed leak demonstration in Part 1 — `ls /tmp/mytool.* | wc -l` growing by one per run until the trap is in — is worth showing the whole cohort: it is the only moment in the course where a learner *sees* a temp leak accumulate.

Part 1: Prepare the exercise folder and watch the leak

Step 1: Run the grader once and count temp files.

```
cd labs/capstone/exercise
bash grade.sh
ls /tmp/mytool.* 2>/dev/null | wc -l
bash grade.sh >/dev/null 2>&1; ls /tmp/mytool.* 2>/dev/null | wc -l
```

Expected: `---- capstone: 2 passed, 10 failed ----`, and a count that grows by one each time you run the grader: the starter creates its temp file but has **no trap**, so every run leaves one behind. You will watch it stop growing once your trap is in. Clean up: `rm -f /tmp/mytool.*`.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Step 2: Read `work/mytool` top to bottom before editing.

The `# TODO` markers are in the order you should fill them: the trap, `usage`, the `getopts` loop, the subcommand check, FILE-or-stdin into the temp file, the `case` dispatch. Each marker says *what* the line must do; the modules showed you *how*.

Part 2: Assemble `mytool`

Work through the markers in order and re-run `bash grade.sh` after each one, reading only the **first** `FAIL` — the checks are ordered so that fixing it usually unlocks the next.

Step 1: The cleanup trap (S07.3–S07.4).

The temp file is already created with `mktemp`. Under the `# shellcheck disable=SC2154` line (keep it), add a trap that captures the exit code first, removes the file, and re-exits with the saved code:

> **Instructor Note:**
>
> ```
> trap 'rc=$?; rm -f "$tmp"; exit "$rc"' EXIT INT TERM
> ```

Check it: run the grader twice and `ls /tmp/mytool.*` — the count no longer grows.

Step 2: `usage` (S06.5).

Make `usage` print a short help text to **stdout** — the command line, the two subcommands, the `-n` option, and the FILE-or-stdin rule. A here-document is the natural form.

> **Instructor Note:**
>
> ```
> usage() {
>   cat <<EOF
> usage: $PROG [-n N] <count|top> [FILE]
>   count        number of lines (FILE or stdin)
>   top          the most frequent line(s)
>   -n N         for 'top', show the top N (default 1)
> EOF
> }
> ```

Step 3: Option parsing (S06.2).

Parse `-n N` and `-h` with `getopts ":n:h"`: `n` stores the value; `h` prints usage and exits 0; `:` (missing argument) and `\?` (unknown option) print a message to stderr and exit **2**. Then `shift $((OPTIND - 1))`.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note:

```
n=1
while getopts ":n:h" opt; do
  case "$opt" in
    n)  n="$OPTARG" ;;
    h)  usage; exit 0 ;;
    :)  printf '%s: -%s needs an argument\n' "$PROG" "$OPTARG" >&2; exit 2 ;;
    \?) printf '%s: unknown option -%s\n' "$PROG" "$OPTARG" >&2; exit 2 ;;
  esac
done
shift $((OPTIND - 1))
```

Step 4: Require a subcommand (S06.5).

If no argument is left, print usage to **stderr** and exit 2. Otherwise take the first argument as the command and shift it away.

Instructor Note:

```
[[ $# -ge 1 ]] || { usage >&2; exit 2; }
cmd="$1"; shift
```

Step 5: Read FILE-or-stdin into the temp file (S06.4).

If a FILE argument remains, check it is readable — if not, print a message to stderr and exit **1** — then copy it into `$tmp` with `cat -- "$1"`. With no FILE, copy stdin into `$tmp`.

Instructor Note:

```
if [[ $# -ge 1 ]]; then
  [[ -r "$1" ]] || { printf '%s: cannot read file: %s\n' "$PROG" "$1" >&2; exit 1; }
  cat -- "$1" > "$tmp"
else
  cat > "$tmp"
fi
```

Step 6: Dispatch (S02.3, S05.3).

A `case` on `$cmd`: `count` counts the lines of `$tmp`; `top` is the counting pipeline from M05 followed by `sed 's/^ *[0-9]* //'` to drop the counts and keep the lines; anything else prints `unknown command` to stderr and exits **2**.

Instructor Note:

```
case "$cmd" in
  count) wc -l < "$tmp" | tr -d ' ' ;;
  top)   sort "$tmp" | uniq -c | sort -rn | head -n "$n" | sed 's/^ *[0-9]* //' ;;
  *)     printf '%s: unknown command: %s\n' "$PROG" "$cmd" >&2; exit 2 ;;
esac
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Step 7: Try it by hand.

```
printf 'apple\nbanana\napple\ncherry\napple\nbanana\n' > /tmp/fruit.txt
bash work/mytool count /tmp/fruit.txt
printf 'a\nb\nc\n' | bash work/mytool count
bash work/mytool -n 2 top /tmp/fruit.txt
bash work/mytool -h | head -1
bash work/mytool frob /tmp/fruit.txt; echo "status=$?"
bash work/mytool count /nope; echo "status=$?"
bash work/mytool -n; echo "status=$?"
rm -f /tmp/fruit.txt
```

Expected: `6`; `3`; `apple` then `banana`; `usage: mytool [-n N] <count|top> [FILE]`; `mytool: unknown command: frob` and `status=2`; `mytool: cannot read file: /nope` and `status=1`; `mytool: -n needs an argument` and `status=2`.

Part 3: Verify

Step 1: shellcheck, then the grader.

```
shellcheck work/mytool
bash grade.sh
bash grade.sh > work/GRADER.txt 2>&1
```

Expected: no shellcheck output, and `---- capstone: 12 passed, 0 failed ----`. The twelve checks, in order: `count` counts lines of a FILE; `count` reads stdin when FILE is omitted; `top` returns the most frequent line; `top -n 2` returns two lines; a good run exits 0; an unknown command exits non-zero; an unknown command exits exactly 2; an unreadable file exits exactly 1; `-h` prints usage and exits 0; runs under strict mode; cleans up with a trap (exactly one line beginning `trap`); shellcheck-clean.

Symptom	Cause	Fix
`-h prints usage and exits 0` fails although `-h` works in your terminal	Usage goes to stderr; the grader runs with `2>/dev/null`	`-h` must print to **stdout**
`unknown command exits exactly 2` fails	The `*)` branch exits 1	Usage mistakes are 2; 1 is for runtime failures
`cleans up with a trap` fails	No `trap` line, or more than one	Exactly one line beginning `trap`
`top` prints counts	The `sed` is missing	`sed 's/^ *[0-9]* //'` after `head`

Step 2: Reset.

`rm -rf work && bash grade.sh`; remove leaked temp files with `rm -f /tmp/mytool.*`.

Part 4: The decisions note

Write `work/DECISIONS.md` — about 150 words in total, in your own words, three short answers. State whether you used AI assistance (documentation lookup only) or none.

D1. Your trap begins with `rc=$?`. Why there, and which of *your* exit codes would be lost without it?

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

> **Answer:** **Met:** `$?` holds the exit status only until the next command runs, so the trap must save it before `rm -f`; under `set -e` a failing cleanup would otherwise turn the exit into `1` and the tool's `2` (usage) or `1`-vs-`0` distinction would be lost. **Strong:** quotes S07.3 (`status=1` vs `status=3`) or names `logsleuth bruteforce`'s `3` as the code this idiom protects.
>
> *Calibration.* Strong: "The trap saves `$?` into `rc` first because every command, including `rm -f`, replaces `$?`. If the cleanup failed under `set -e` the script would exit 1 — in M07 I saw `with -e: status=1` versus `status=3` once the code was captured — so my `exit 2` for a bad option would come out as 1 and a wrapper could not tell a typo from a missing file." Met: "`rc=$?` saves the real exit code before cleanup so that `exit "$rc"` can return it; otherwise a failing `rm` would change it." Not met: "You have to save the return code so the script returns the right code."

D2. `mytool -h` prints usage to stdout and exits 0; a usage *mistake* prints to stderr and exits 2, not 1. Who depends on each of those two choices?

> **Answer:** **Met:** `-h` output *is* the requested result, so it belongs on stdout where `mytool -h | less` or a caller can read it; an error message must not pollute captured output, so it goes to stderr; exit 2 tells a caller (a wrapper, cron, `||`) that the *command line* was wrong rather than that a file was unreadable (1). **Strong:** gives a concrete consumer for each (`out=$(mytool …)` for stderr; `mytool … || alert` or a CI step for the code), or cites the grader's `-h 2>/dev/null | grep -q usage` as the reason stdout matters.

D3. `mytool` copies its input (file or stdin) into a `mktemp` file before running `wc`/`sort` on it. What does the temp file buy you, and what does the trap then owe in return?

> **Answer:** **Met:** the temp file lets the tool treat stdin and a file identically and read the data once (stdin cannot be rewound) so `count` and `top` share one path; in return the trap must remove the file on every exit, including signals, or the tool leaks a file per run (observed with the starter). **Strong:** names `mktemp`'s uniqueness (no collisions between concurrent runs) or the `TMPDIR` convention, or notes the alternative (`/dev/stdin`) and its trade-off.

Rubric R-10 and hand-in

Hand in `work/mytool`, `work/DECISIONS.md` and `work/GRADER.txt`. Each criterion is judged **Not met** (0), **Met** (1) or **Strong** (2) and scored `weight × level / 2`. **Pass** = total ≥ 80 % of 20 **and** no *critical* criterion at Not met. *Strong* means "built to the standard the modules taught"; *Met* means "works, with a deviation the modules warned about"; *Not met* means a check fails.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

ID	Criterion	Evidence	Weight	Critical
R-10.1	Functional: `count` (file and stdin), `top`, `top -n N`	checks 1–4	5	—
R-10.2	Exit codes are a correct API: 0 ok, 1 unreadable file, 2 usage / unknown command	checks 5–8	3	**yes**
R-10.3	`-h` prints usage to stdout and exits 0	check 9	1	—
R-10.4	Strict-mode header	check 10	1	—
R-10.5	Cleanup trap present and code-preserving	check 11 + the trap line	2	**yes**
R-10.6	shellcheck-clean	check 12	2	**yes**
R-10.7	Decisions note: three answers that name the mechanism in your own words	`DECISIONS.md`	6	—

A green grader with no decisions note scores 14/20 = 70 % — **not a pass**. The note is where you show the understanding the grader cannot see.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note: Level descriptors.

- **R-10.1 (w 5).** *Not met:* any of checks 1–4 fails except as below. *Met:* exactly one of the four fails **or** all pass but input handling deviates from the taught form (`cat` without `--`, no readability check before reading). *Strong:* all four pass with `cat -- "$1"` guarded by a readability test.
- **R-10.2 (w 3, critical).** *Not met:* any of checks 5–8 fails. *Met:* all pass but a usage-class error (missing `-n` argument, unknown option) exits with a code other than 2, or error text goes to stdout. *Strong:* every usage-class error exits 2 with a message on stderr; runtime failure exits 1.
- **R-10.3 (w 1).** *Not met:* check 9 fails. *Met:* passes but usage lacks the subcommands or `-n`. *Strong:* usage names both subcommands, `-n`, and the FILE-or-stdin rule.
- **R-10.4 (w 1).** *Not met:* check 10 fails. *Met:* header present but not within the first lines. *Strong:* header at the top, as in M00.
- **R-10.5 (w 2, critical).** *Not met:* check 11 fails (no trap line, or more than one). *Met:* a trap removes the temp file but does not capture and re-exit `$?`. *Strong:* `trap 'rc=$?; rm -f "$tmp"; exit "$rc"' EXIT INT TERM` or an equivalent capture-and-re-exit — observed to preserve codes 0/1/2.
- **R-10.6 (w 2, critical).** *Not met:* any finding (info counts). *Met:* clean only with a `# shellcheck disable=` other than the course's `SC2154` line. *Strong:* clean with at most the `SC2154` disable the starter provides.
- **R-10.7 (w 6).** Scored on the three answers together. *Not met:* missing, under ~60 words, or answers that restate the question, name no mechanism, or contradict the submitted code. *Met:* each answer names the mechanism (model answers above) in the learner's own words. *Strong:* at least two answers go beyond — an observation from the modules, a concrete failure the choice prevents, or a correct link to `logsleuth`/M09 — and the AI-use disclosure is present (or an explicit "none").

Deviations and how they score. `trap 'rm -f "$tmp"' EXIT` — passes check 11, R-10.5 Met. `exit 1` for an unknown command — fails check 7, R-10.2 Not met → **fails the assessment** (critical). Usage to stderr for `-h` — fails check 9. `cat "$1"` without `--` — passes everything, R-10.1 Met. `#!/bin/bash` — passes (the grader does not check the shebang); note it in feedback. An implementation that skips the temp file and reads `"${1:-/dev/stdin}"` directly can pass all 12 checks if a trap line is still present; it is **not** a deviation to penalise — score it on its merits and expect D3 to reason about its *own* design.

Scoring procedure (≈12 min per learner). 1) Re-run the grader on their copy. 2) Read `mytool` for the Strong/Met distinctions (five minutes: trap line, `--`, where usage goes, exit codes in the error branches, shellcheck disables). 3) Read the note against the model answers. 4) Compute Σ(weight × level / 2) / 20. 5) **Pass** only if ≥ 80 % **and** R-10.2, R-10.5, R-10.6 are each ≥ Met. 6) Record `A-10 PASS` or `A-10 NOT YET` with the per-criterion levels — that record is the remediation plan.

Worked examples (observed arithmetic). All Strong → 20/20. Everything Strong but no note → 14/20 = 70 %, fail. All Strong, note Met → 17/20 = 85 %, pass. R-10.1 Met (`cat` without `--`), R-10.5 Met (bare `rm` trap), note Strong → 2.5 + 3 + 1 + 1 + 1 + 2 + 6 = 16.5/20 = 82.5 %, pass — with two deviations to feed back. `exit 1` for an unknown command, everything else Strong → 17/20 = 85 % **but R-10.2 Not met → fail** (critical gate).

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Model feedback (Not yet): "`unknown command exits exactly 2` FAILED: your `*)` branch exits 1 — the course reserves 1 for runtime failures; usage mistakes are 2 (S06.5). This is a critical criterion, so A-10 is Not yet regardless of the rest. Remediation: S06.5 and S02.4, then A-10R."
Remediation map: R-10.1 → S06.4, S05.3; R-10.2 → S06.5; R-10.3 → S06.5; R-10.4 → S00.7; R-10.5 → S07.3–S07.4; R-10.6 → S00.9, S08.3; R-10.7 → re-read the relevant step and rewrite in own words.

A-10R — equivalent reassessment (kit in `trainer/a10r/`). Same 12-check shape, same weights, levels and critical gates; different names and codes so the first attempt cannot be pasted in: `linetool lines [FILE]`, `linetool [-k N] freq [FILE]`, `linetool -h`; exit map 0 / **3** (unreadable file) / **4** (usage or unknown command). The starter `trainer/a10r/starter/linetool` states only the contract — no line-by-line hints — so A-10R also evidences independent assembly. Grade with `bash trainer/a10r/grade-A10R.sh /path/to/their/linetool` → require `---- A-10R: 12 passed, 0 failed ----` (observed: reference 12/12, contract-only starter 2/12; the A-10 grader scores the A-10R reference only 5/12, confirming the variant is distinct). Use the same three decision prompts with the names and codes substituted; `TASK-A10R.md` in the kit is the hand-out.

Evidence handling. Keep `mytool`, `DECISIONS.md`, your re-run grader output and the per-criterion levels. Reset for the next session: `rm -rf work; rm -f /tmp/mytool.*`.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note: Appendix — reference solution (`solution/mytool`, observed 12/12):

```
#!/usr/bin/env bash
# Capstone solution — mytool: a tiny line-analyzer that scores every row of the
# rubric logsleuth meets (strict mode, usage, getopts, case dispatch, file-or-
# stdin, distinct exit codes, a cleanup trap).
set -euo pipefail
PROG="mytool"

tmp="$(mktemp "${TMPDIR:-/tmp}/$PROG.XXXXXX")"
# shellcheck disable=SC2154  # rc is assigned inside the trap string, then used
trap 'rc=$?; rm -f "$tmp"; exit "$rc"' EXIT INT TERM

usage() {
  cat <<EOF
usage: $PROG [-n N] <count|top> [FILE]
  count        number of lines (FILE or stdin)
  top          the most frequent line(s)
  -n N         for 'top', show the top N (default 1)
EOF
}

n=1
while getopts ":n:h" opt; do
  case "$opt" in
    n)  n="$OPTARG" ;;
    h)  usage; exit 0 ;;
    :)  printf '%s: -%s needs an argument\n' "$PROG" "$OPTARG" >&2; exit 2 ;;
    \?) printf '%s: unknown option -%s\n' "$PROG" "$OPTARG" >&2; exit 2 ;;
  esac
done
shift $((OPTIND - 1))

[[ $# -ge 1 ]] || { usage >&2; exit 2; }
cmd="$1"; shift

# read FILE-or-stdin into the temp file
if [[ $# -ge 1 ]]; then
  [[ -r "$1" ]] || { printf '%s: cannot read file: %s\n' "$PROG" "$1" >&2; exit 1; }
  cat -- "$1" > "$tmp"
else
  cat > "$tmp"
fi

case "$cmd" in
  count) wc -l < "$tmp" | tr -d ' ' ;;
  top)   sort "$tmp" | uniq -c | sort -rn | head -n "$n" | sed 's/^ *[0-9]* //' ;;
  *)     printf '%s: unknown command: %s\n' "$PROG" "$cmd" >&2; exit 2 ;;
esac
```

Observed behaviour of the reference: `-h` → four usage lines on stdout, exit 0; no arguments → usage on stderr, exit 2; `-n` alone → `mytool: -n needs an argument`, exit 2; `-x` → `unknown option -x`, exit 2; `count /nope` → `cannot read file: /nope`, exit 1; `frob FILE` → `unknown command: frob`, exit 2; `-n 2 top` on the grader's data → `apple` then `banana`; `printf 'a\nb\nb\n' | mytool top` → `b`. After any number of runs `ls /tmp/mytool.*` is unchanged.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.